SQL SERVER — RÉVISION GÉNÉRALE

Base REVISION_DB

WHERE · Filtres · Sous-requêtes · CTE · Index · Recovery Model · Sauvegardes

Jeudi 22 octobre 2026 • Base : REVISION_DB • SQL Server 2022

★ **CONCEPT CLÉ** OBJECTIFS DE LA SESSION

1. Créer une base complète avec tables liées (FK).
2. Maîtriser WHERE, AND/OR, IN, BETWEEN, LIKE, IS NULL.
3. Comprendre GROUP BY, HAVING et l'ordre d'exécution SQL.
4. Utiliser les sous-requêtes et les CTE.
5. Créer et gérer les index — clustered vs non-clustered.
6. Réviser Recovery Model et stratégie de sauvegarde.

1. Création de la Base REVISION_DB

La base REVISION_DB contient deux tables liées : Departements et Employes. La clé étrangère FK_Employes_Dept garantit l'intégrité référentielle.

```
USE master;
GO

CREATE DATABASE REVISION_DB;
GO

USE REVISION_DB;
GO

-- Table parent
CREATE TABLE dbo.Departements
(
    DepartementID   INT IDENTITY(1,1) NOT NULL,
    Nom             NVARCHAR(100)     NOT NULL,
    Localisation    NVARCHAR(100)         NULL,
    DateCreation    DATETIME2         NOT NULL DEFAULT SYSDATETIME(),

    CONSTRAINT PK_Departements     PRIMARY KEY CLUSTERED (DepartementID),
    CONSTRAINT UQ_Departements_Nom UNIQUE (Nom)
);
GO

-- Table enfant avec FK
CREATE TABLE dbo.Employes
(
    EmployeID       INT IDENTITY(1,1) NOT NULL,
    Nom             NVARCHAR(100)     NOT NULL,
    Prenom          NVARCHAR(100)     NOT NULL,
    Email           NVARCHAR(200)     NOT NULL,
    Salaire         DECIMAL(10,2)     NOT NULL,
    DepartementID   INT               NOT NULL,
    DateEmbauche    DATETIME2         NOT NULL DEFAULT SYSDATETIME(),

    CONSTRAINT PK_Employes       PRIMARY KEY CLUSTERED (EmployeID),
    CONSTRAINT UQ_Employes_Email UNIQUE (Email),
    CONSTRAINT FK_Employes_Dept  FOREIGN KEY (DepartementID)
        REFERENCES dbo.Departements (DepartementID)
```

```
);
GO
```

⚠ **ATTENTION** Ordre de création obligatoire avec FK
La table parent (Departements) doit être créée AVANT la table enfant (Employes).
Pour supprimer : DROP TABLE dbo.Employes d'abord, puis dbo.Departements.
Sinon SQL Server retourne une erreur de contrainte FK.

2. Filtres WHERE — Tous les Opérateurs

2.1 Filtres simples

```
-- Filtre exact
SELECT * FROM dbo.Employes WHERE DepartementID = 1;

-- Comparaison numérique
SELECT * FROM dbo.Employes WHERE Salaire > 4000;

-- Intervalle (inclusif des deux bornes)
SELECT * FROM dbo.Employes WHERE Salaire BETWEEN 3000 AND 4500;

-- Recherche par motif
SELECT * FROM dbo.Employes WHERE Nom LIKE N'B%';

-- Valeur nulle
SELECT * FROM dbo.Employes WHERE DepartementID IS NULL;

-- Liste de valeurs
SELECT * FROM dbo.Employes WHERE DepartementID IN (1, 2);
```

2.2 Filtres combinés AND / OR

```
-- AND : les deux conditions doivent être vraies
SELECT * FROM dbo.Employes
WHERE DepartementID = 1 AND Salaire > 3000;

-- OR : au moins une condition vraie
SELECT * FROM dbo.Employes
WHERE DepartementID = 2 OR Salaire > 4000;

-- Parenthèses obligatoires pour grouper OR avec AND
SELECT * FROM dbo.Employes
WHERE (DepartementID IN (1, 3)) AND Salaire > 3000;
```

⚠ **ATTENTION** Priorité des opérateurs : AND avant OR
WHERE A OR B AND C → SQL lit : A OR (B AND C)
Toujours mettre des parenthèses pour éviter les surprises.
Exemple : WHERE (dept=1 OR dept=3) AND salaire > 3000

2.3 WHERE avec JOIN

```
-- Filtrer par nom de département (pas par ID)
SELECT e.Nom, e.Prenom, e.Salaire, d.Nom AS Departement
FROM dbo.Employes e
JOIN dbo.Departements d ON e.DepartementID = d.DepartementID
WHERE d.Nom = N'Informatique';

-- Filtre sur deux tables
SELECT e.Nom, e.Prenom, e.Salaire, d.Localisation
FROM dbo.Employes e
JOIN dbo.Departements d ON e.DepartementID = d.DepartementID
WHERE d.Localisation = N'Paris'
AND e.Salaire > 3000;
```

2.4 Filtre optionnel — Pattern production

Utilisé dans les procédures stockées pour rendre les paramètres optionnels.

```
DECLARE @Nom           NVARCHAR(100) = NULL;
DECLARE @DepartementID INT           = 1;

SELECT e.EmployeID, e.Nom, e.Prenom, e.Salaire, d.Nom AS Departement
FROM dbo.Employes e
JOIN dbo.Departements d ON e.DepartementID = d.DepartementID
WHERE
    (@Nom           IS NULL OR e.Nom           = @Nom)
    AND
    (@DepartementID IS NULL OR e.DepartementID = @DepartementID)
ORDER BY e.Nom;
```

★ **CONCEPT CLÉ** Logique du filtre optionnel
Si @Nom est NULL → (@Nom IS NULL) est VRAI → court-circuit → pas de filtre sur Nom.
Si @Nom a une valeur → (@Nom IS NULL) est FAUX → SQL évalue (Nom = @Nom) → filtre actif.

3. GROUP BY, HAVING et Ordre d'Exécution

3.1 GROUP BY avec agrégats

```
-- Salaire moyen par département
SELECT
    d.Nom           AS Departement,
    COUNT(e.EmployeID) AS NombreEmployes,
    AVG(e.Salaire)  AS SalaireMoyen,
    MIN(e.Salaire)  AS SalaireMin,
    MAX(e.Salaire)  AS SalaireMax
FROM dbo.Employes e
JOIN dbo.Departements d ON e.DepartementID = d.DepartementID
GROUP BY d.Nom
ORDER BY SalaireMoyen DESC;
```

3.2 HAVING — Filtrer les groupes

```
-- Départements dont la moyenne salariale dépasse 3500
SELECT
    d.Nom          AS Departement,
    AVG(e.Salaire) AS SalaireMoyen
FROM dbo.Employes e
JOIN dbo.Departements d ON e.DepartementID = d.DepartementID
GROUP BY d.Nom
HAVING AVG(e.Salaire) > 3500
ORDER BY SalaireMoyen DESC;
```

	WHERE	HAVING
S'applique sur	Les lignes de la table	Les groupes créés par GROUP BY
S'exécute	Avant le GROUP BY	Après le GROUP BY
Peut utiliser	Colonnes normales	Fonctions d'agrégat (COUNT, AVG…)

🎯 **QUESTION D'ENTRETIEN** Ordre d'exécution SQL — À retenir par cœur
FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
WHERE filtre les lignes AVANT le regroupement.
HAVING filtre les groupes APRÈS le regroupement.
On ne peut pas utiliser AVG() ou COUNT() dans un WHERE.

4. Sous-requêtes

Une sous-requête est une requête imbriquée dans une autre. Elle permet de comparer chaque ligne à une valeur calculée dynamiquement.

4.1 Sous-requête dans WHERE

```
-- Employés qui gagnent plus que la moyenne
SELECT
    Nom,
    Prenom,
    Salaire,
    (SELECT AVG(Salaire) FROM dbo.Employes) AS SalaireMoyen
FROM dbo.Employes
WHERE Salaire > (SELECT AVG(Salaire) FROM dbo.Employes)
ORDER BY Salaire DESC;
```

4.2 Sous-requête dans SELECT

```
-- Écart entre le salaire et la moyenne
SELECT
    Nom,
    Prenom,
    Salaire,
    ROUND(Salaire - (SELECT AVG(Salaire) FROM dbo.Employes), 2) AS EcartMoyenne
FROM dbo.Employes
ORDER BY EcartMoyenne DESC;
```

4.3 Sous-requête avec IN

```
-- Employés dans les départements de Paris ou Lyon
SELECT Nom, Prenom, Salaire
FROM dbo.Employes
WHERE DepartementID IN (
    SELECT DepartementID
    FROM dbo.Departements
    WHERE Localisation IN (N'Paris', N'Lyon')
)
ORDER BY Salaire DESC;
```

🎯 **QUESTION D'ENTRETIEN** Valeur fixe vs sous-requête — Question d'entretien classique
WHERE Salaire > 3500 → valeur fixe, obsolète si les données changent.
WHERE Salaire > (SELECT AVG(Salaire)...) → dynamique, toujours à jour.
La sous-requête s'adapte automatiquement aux données actuelles.

5. CTE — Common Table Expression

La CTE est la version moderne et lisible de la sous-requête. Elle se déclare avec WITH avant la requête principale.

5.1 CTE simple

```
WITH CTE_SalaireMoyen AS
(
    SELECT AVG(Salaire) AS Moyenne
    FROM dbo.Employes
)
SELECT
    e.Nom,
    e.Prenom,
    e.Salaire,
    c.Moyenne       AS SalaireMoyen,
    ROUND(e.Salaire - c.Moyenne, 2) AS Ecart
FROM dbo.Employes e
CROSS JOIN CTE_SalaireMoyen c
WHERE e.Salaire > c.Moyenne
ORDER BY e.Salaire DESC;
```

5.2 CTE multiple — Version production

```
WITH
MoyenneSalaire AS (
    SELECT AVG(Salaire) AS Moyenne
    FROM dbo.Employes
),
DeptParis AS (
    SELECT DepartementID
    FROM dbo.Departements
    WHERE Localisation = N'Paris'
)
SELECT e.Nom, e.Salaire
FROM dbo.Employes e
JOIN DeptParis d ON e.DepartementID = d.DepartementID
CROSS JOIN MoyenneSalaire m
WHERE e.Salaire > m.Moyenne;
```

Critère	Sous-requête	CTE
Syntaxe	Imbriquée dans la requête	Déclarée avant avec WITH
Lisibilité	Difficile si complexe	Claire et structurée
Réutilisation	Non — répétée à chaque fois	Oui — définie une fois
Débogage	Difficile à isoler	Facile — testable séparément
Performance	Similaire en général	Similaire, parfois mieux

✔ **BONNE PRATIQUE DBA** En production, préfère les CTE pour tout ce qui dépasse une ligne. Le code est plus lisible, plus facile à maintenir, et plus facile à déboguer.

6. Index — Optimisation des Performances

Un index est une structure de données qui permet à SQL Server de trouver rapidement les lignes sans scanner toute la table.

Type	Description	Limite
CLUSTERED	Trie physiquement les données	1 seul par table
NONCLUSTERED	Structure séparée pointant vers les données	Jusqu'à 999 par table
UNIQUE	Garantit l'unicité des valeurs	Clustered ou Non-Clustered
FILTERED	Index partiel sur une condition WHERE	Non-Clustered uniquement

ℹ️ **INFO** Index créés automatiquement par SQL Server
PRIMARY KEY → crée un index CLUSTERED automatiquement.
Contrainte UNIQUE → crée un index NONCLUSTERED automatiquement.
Tu n'as pas besoin de les créer manuellement.

6.1 Créer des index

```
-- Index sur Email (recherche exacte fréquente)
CREATE NONCLUSTERED INDEX IX_Employes_Email
ON dbo.Employes (Email);
GO

-- Index sur DepartementID (jointures fréquentes)
CREATE NONCLUSTERED INDEX IX_Employes_DepartementID
ON dbo.Employes (DepartementID);
GO

-- Index sur Salaire trié DESC (ORDER BY et filtres)
CREATE NONCLUSTERED INDEX IX_Employes_Salaire
ON dbo.Employes (Salaire DESC);
GO
```

6.2 Vérifier les index

```
SELECT
    i.name          AS IndexName,
    i.type_desc     AS TypeIndex,
    c.name          AS Colonne,
    i.is_unique     AS EstUnique
FROM sys.indexes i
JOIN sys.index_columns ic ON i.object_id = ic.object_id
    AND i.index_id = ic.index_id
JOIN sys.columns c ON ic.object_id = c.object_id
    AND ic.column_id = c.column_id
WHERE i.object_id = OBJECT_ID('dbo.Employes')
ORDER BY i.index_id;
GO
```

6.3 Fragmentation et maintenance

```
-- Vérifier la fragmentation
SELECT
    i.name                          AS IndexName,
    s.avg_fragmentation_in_percent AS Fragmentation,
    s.page_count                    AS NombrePages
FROM sys.dm_db_index_physical_stats(
    DB_ID(), OBJECT_ID('dbo.Employes'), NULL, NULL, 'LIMITED') s
JOIN sys.indexes i ON s.object_id = i.object_id
    AND s.index_id = i.index_id
WHERE i.name IS NOT NULL
ORDER BY s.avg_fragmentation_in_percent DESC;
GO

-- Réorganiser (fragmentation 10-30%)
ALTER INDEX IX_Employes_Email ON dbo.Employes REORGANIZE;
GO

-- Reconstruire (fragmentation > 30%)
ALTER INDEX IX_Employes_Email ON dbo.Employes REBUILD;
GO
```

Fragmentation	Action recommandée	Impact
< 10%	Rien à faire	Normal
10% — 30%	REORGANIZE	Rapide, opération en ligne
> 30%	REBUILD	Complet, plus long, verrou possible

🎯 **QUESTION D'ENTRETIEN** Question d'entretien — Table lente sur WHERE Email = '...'
Cause : Full Table Scan — SQL lit toutes les lignes.
Solution : CREATE NONCLUSTERED INDEX IX_Employes_Email ON dbo.Employes (Email).
Résultat : SQL utilise l'index → accès direct → millisecondes au lieu de secondes.

7. Révision — Recovery Model et Sauvegardes

7.1 Recovery Models

Mode	Journal	BACKUP LOG	PITR	Usage
FULL	Toutes les transactions	Oui	Oui	Production critique
BULK_LOGGED	Opérations bulk minimales	Oui	Limité	Chargements massifs
SIMPLE	Tronqué au checkpoint	Non	Non	Dev / Test

⚠ **ATTENTION** Nuance critique — FULL ne suffit pas
FULL Recovery Model ne signifie pas 'je peux faire des BACKUP LOG'.
Il signifie 'je peux démarrer une chaîne de logs APRÈS un premier FULL backup'.
Sans premier BACKUP DATABASE complet → BACKUP LOG échoue.

7.2 Ordre de restauration

```
-- 1. Toujours commencer par le FULL
RESTORE DATABASE MaBase
FROM DISK = N'...MaBase_FULL.bak'
WITH NORECOVERY, CHECKSUM, STATS = 10;
GO

-- 2. Appliquer le DIFF (seulement le dernier)
RESTORE DATABASE MaBase
FROM DISK = N'...MaBase_DIFF_002.bak'
WITH NORECOVERY, CHECKSUM, STATS = 10;
GO

-- 3. Appliquer les LOGs dans l'ordre
RESTORE LOG MaBase
FROM DISK = N'...MaBase_LOG_001.trn'
WITH NORECOVERY, CHECKSUM, STATS = 10;
GO

-- 4. Appliquer le TAIL LOG avec STOPAT
RESTORE LOG MaBase
FROM DISK = N'...MaBase_TAIL.trn'
WITH STOPAT = N'2026-10-22T14:00:00',
     RECOVERY, CHECKSUM, STATS = 10;
GO
```

Option	État de la base	Quand l'utiliser
NORECOVERY	RESTORING	Tous les RESTORE sauf le dernier — d'autres fichiers à appliquer
RECOVERY	ONLINE	Dernier RESTORE — ouvre la base, phase UNDO exécutée
STANDBY	Lecture seule	Pour permettre des SELECT pendant la restauration

🎯 **QUESTION D'ENTRETIEN** Pourquoi NORECOVERY sur tous sauf le dernier ?
NORECOVERY laisse la base en état RESTORING → SQL peut encore recevoir des fichiers.
RECOVERY exécute la phase UNDO (annule les transactions non validées) → base ONLINE.
Si on met RECOVERY trop tôt → la base s'ouvre → impossible d'appliquer d'autres LOGs.

8. Récapitulatif Général — Questions d'Entretien

Question	Réponse courte
Différence WHERE vs HAVING ?	WHERE filtre les lignes avant GROUP BY. HAVING filtre les groupes après. AVG/COUNT interdit dans WHERE.
Différence sous-requête vs CTE ?	CTE = sous-requête nommée avec WITH. Plus lisible, réutilisable, facile à déboguer. Préférer en production.
Table lente sur WHERE Email = '...' ?	Full Table Scan. Solution : CREATE NONCLUSTERED INDEX sur Email.
Différence CLUSTERED vs NONCLUSTERED ?	CLUSTERED trie physiquement les données (1 seul). NONCLUSTERED = structure séparée (999 max).
Quand REORGANIZE vs REBUILD ?	10-30% fragmentation → REORGANIZE (rapide). >30% → REBUILD (complet).
Pourquoi NORECOVERY sauf le dernier ?	NORECOVERY laisse la base RESTORING pour appliquer d'autres fichiers. RECOVERY = phase UNDO = base ONLINE.
FULL Recovery ne suffit pas — pourquoi ?	Il faut d'abord un BACKUP DATABASE complet pour démarrer la chaîne de logs.
Ordre de restauration ?	FULL → DIFF (dernier seulement) → LOGs dans l'ordre → TAIL LOG + STOPAT → RECOVERY.
SCOPE_IDENTITY() vs @@IDENTITY ?	SCOPE_IDENTITY() = ID de la portée courante. @@IDENTITY = dernier ID global (dangereux avec triggers).
@@TRANCOUNT > 0 avant ROLLBACK — pourquoi ?	Si BEGIN TRANSACTION jamais atteint, @@TRANCOUNT = 0. ROLLBACK sans transaction → nouvelle erreur.